

MCP

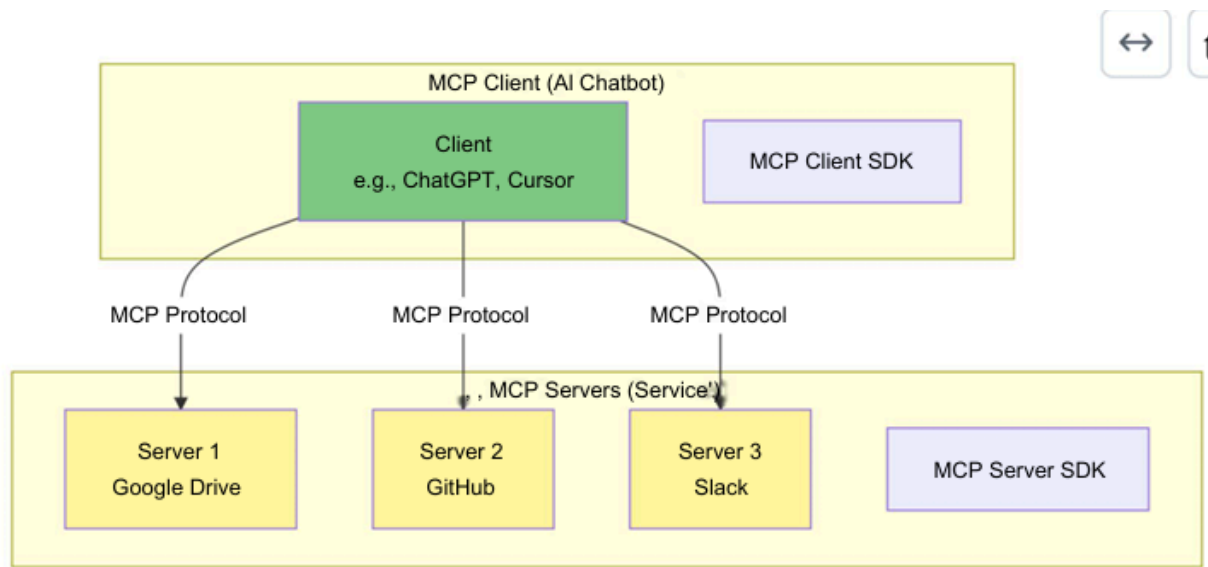
What is MCP?

Model Context Protocol (MCP) is a standardized protocol for communication between AI chatbots (clients) and external services (servers), enabling seamless integration without custom code on the client side.

Full Form Breakdown

- M - Model - Refers to AI/LLM Model
- C - Context - The information AI need to function
- P - Protocol - Standard Rule For Communication

MCP Architecture



Core Component

Term	Meaning	Example
MCP Client	AI chatbot that consumes services through MCP	ChatGPT, Cursor, Perplexity
MCP Server	Service that exposes capabilities through MCP	GitHub server, Slack server
MCP Protocol	Standardized communication rules that clients and servers both speak	MCP standard

MCP Client SDK	Library used to make an AI chatbot MCP-compatible	Client-side SDK
MCP Server SDK	Library used by service providers to create MCP servers	Server-side SDK

MCP vs Function/Tool Calling

Traditional Function Calling

In traditional function calling, **both the client and server require custom code.**

Server Side - API

```
@app.route("/weather")
def weather_endpoint():
    city = request.args.get("city")

    # Query internal weather database
    result = lookup_weather(city)

    return jsonify(result)
```

Client Side — AI Chatbot

```
def get_weather(city: str):
    url = f"https://myweatherapi.com/weather?city={city}&key=API_KEY"

    response = http_get(url)
    data = json_parse(response)

    return f'{city}: {data["temp_c"]}°C, {data["condition"]}'
```

AI Chatbot → Client Code → HTTP Request → Server Code → API/Database

The chatbot developer needs to write and maintain the `get_weather()` function and the API integration.

MCP Approach

With MCP (Model Context Protocol), the MCP server does most of the integration work.

Server Side — MCP Server

```

from mcp.server import Server

server = Server("WeatherServer")

@server.tool("get_weather")
def get_weather(city: str):
    data = lookup_weather(city)

    return {
        "temperature": data["temp_c"],
        "condition": data["condition"]
    }

server.start()

```

Client Side — AI Chatbot

The AI client simply calls the MCP tool:

```
result = call_tool("get_weather", {"city": "London"})
```

MCP Approach

AI Chatbot → MCP Client → MCP Protocol → MCP Server → API/Database

The client doesn't need to know the details of how the weather service works. The MCP server exposes `get_weather` as a standardized tool.

Benefits of MCP

1. **Reduced Integration Complexity** - $M \times N \rightarrow M+N$
2. No Maintenance Overhead - API Change = Server Problem, Single Point of Maintenance, Client maintains 0 integration.
3. Reduced Cost and Time
4. Better Security - Centralized Configuration

Key Problems and Solutions

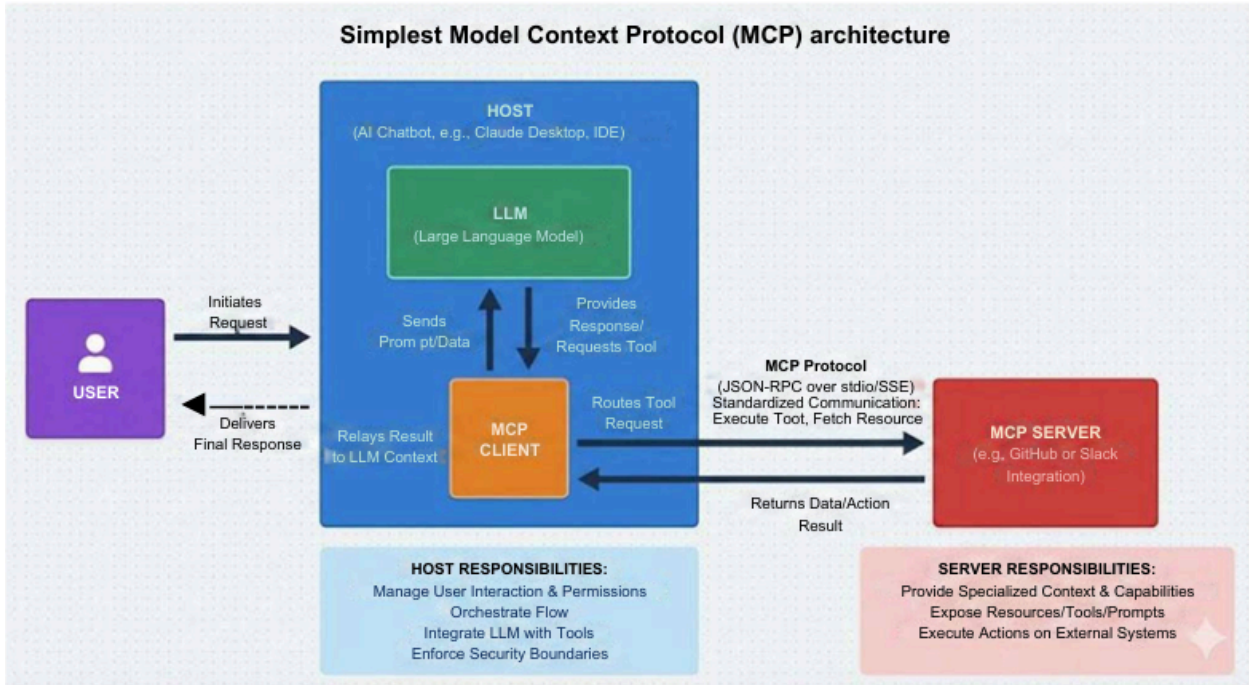
Problem	Description	Solution
Fragmentation	AI tools can't easily talk to each other	Unified protocol
Context Scattering	Information is spread across multiple systems	Tool-based context assembly

Copy-Paste Hell	Manual context assembly is required	Function calling / MCP tools
$N \times M$ Integrations	Integration complexity grows rapidly as systems increase	$M + N$ integrations with MCP
Maintenance Burden	Ongoing integration upkeep becomes difficult	Server-side responsibility

The Core MCP Insight

- The fundamental shift in MCP is that the SERVER does all the heavy lifting. The CLIENT just needs to connect and speak the same language.

MCP Architecture



Note: One-on-One Relationship: An MCP Client can only connect to and communicate with ONE Server at a time. For multiple servers, multiple clients are required.

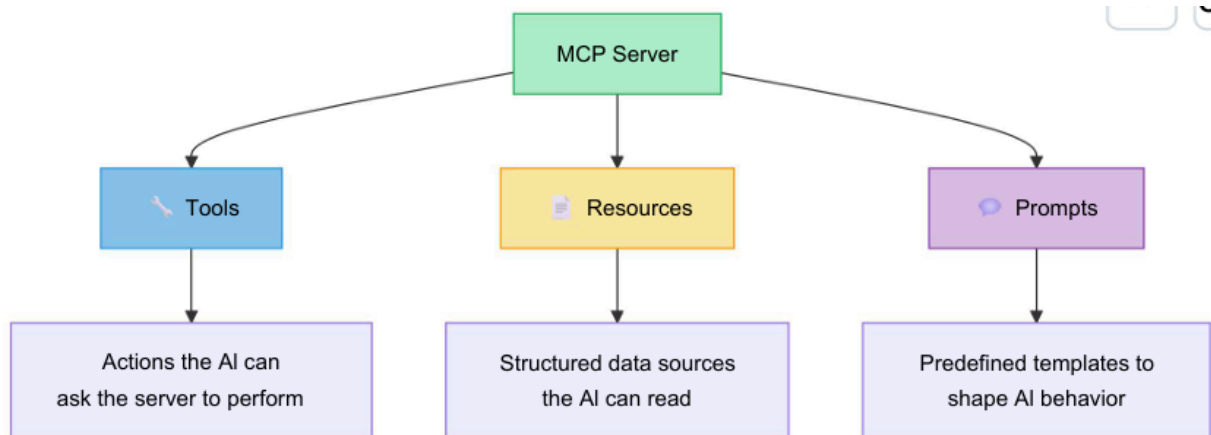
Benefits of MCP Architecture

- Decoupling (Separation of Concerns)
- Scalability
- Parallel Execution

MCP Primitives

Primitives are the offerings or capabilities that a Server provides to the Host. They define what the Server can do.

Every MCP Server offers three types of primitives:



MCP Data Layer

- The Data Layer is the language and grammar of the MCP ecosystem that everyone agrees upon to communicate. It defines HOW messages are structured and formatted.

What is JSON-RPC?

- JSON-RPC (JavaScript Object Notation - Remote Procedure Call) is a protocol that combines Remote Procedure Calls with JSON formatting, allowing structured communication between distributed systems.
- RPC allows a program to execute a function on another computer as if it were local, hiding the details of network communication.

Why JSON-RPC in MCP?

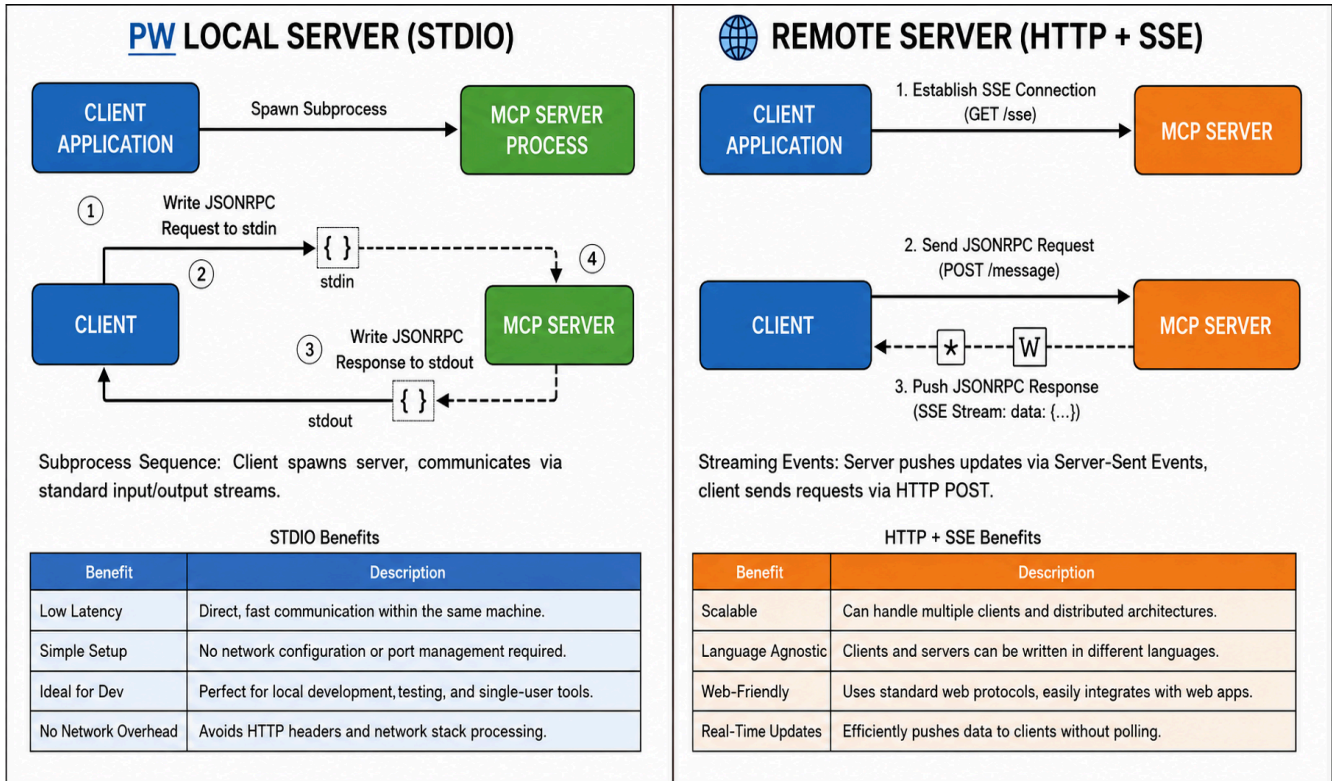
1. Standardized Format: Everyone follows the same rules
2. Language Agnostic: Works with any programming language
3. Human Readable: Easy to debug and understand
4. Widely Supported: Mature protocol with extensive tooling

Why JSON-RPC for Data Layers?

Anthropic chose JSON-RPC 2.0 as the communication protocol for MCP because it provides features that are particularly useful for AI-to-tool communication.

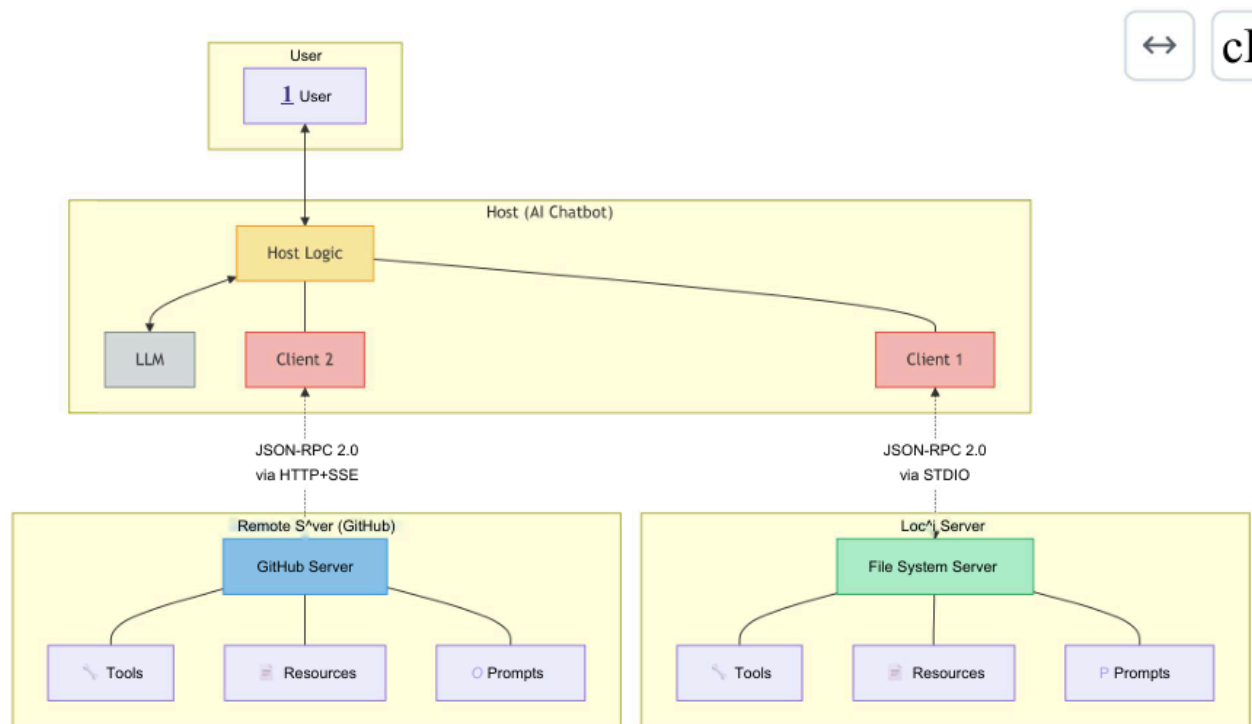
Feature	JSON-RPC 2.0	REST API
Lightweight	Minimal structure	Heavier due to HTTP headers, metadata, etc.
Direction	Bi-directional	Primarily one-way: Client → Server
Transport	Transport-agnostic	HTTP-bound
Batching	Supported	Not natively supported
Notifications	Supported	Not natively supported

Types of MCP Servers



- **Local Server** is an MCP server running on the same computer as the Host application
- **Remote Server** is an MCP server running on a different computer, accessible over a network or the internet.

Final Architecture Diagram



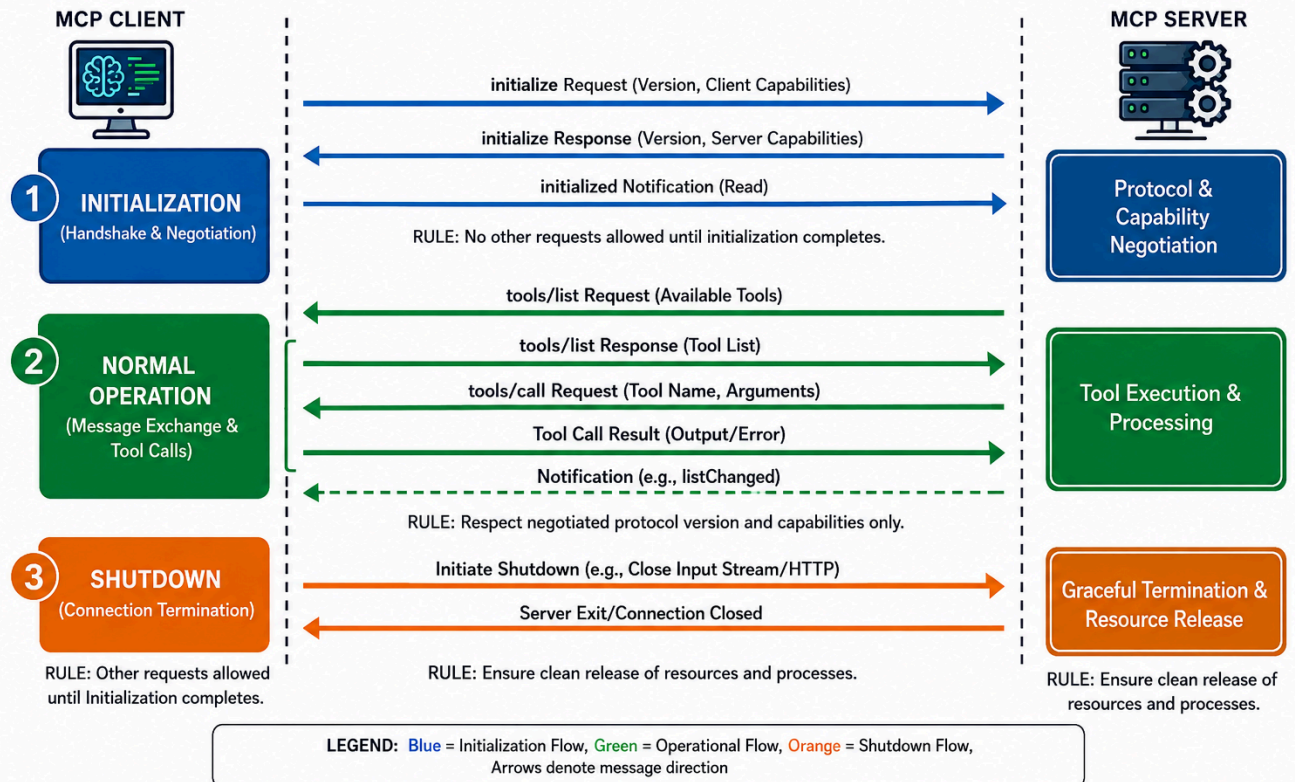
What is MCP Lifecycle?

The MCP Lifecycle describes the complete sequence of steps that govern how a Host (Client) and a Server establish, use, and end a connection during a session.

Session: One continuous connection between the Client and the Server

The MCP Lifecycle is divided into **three** distinct stages:

MCP LIFECYCLE: INITIALIZATION, OPERATION, SHUTDOWN



RAG

What is RAG?

RAG (Retrieval-Augmented Generation) is an AI architecture pattern that enhances Large Language Models (LLMs) by connecting them to external knowledge sources at inference time — rather than relying solely on what the model learned during training.

What Problem Does RAG Solve?

RAG (Retrieval-Augmented Generation) addresses several fundamental limitations of LLMs by retrieving relevant information from external sources before generating a response.

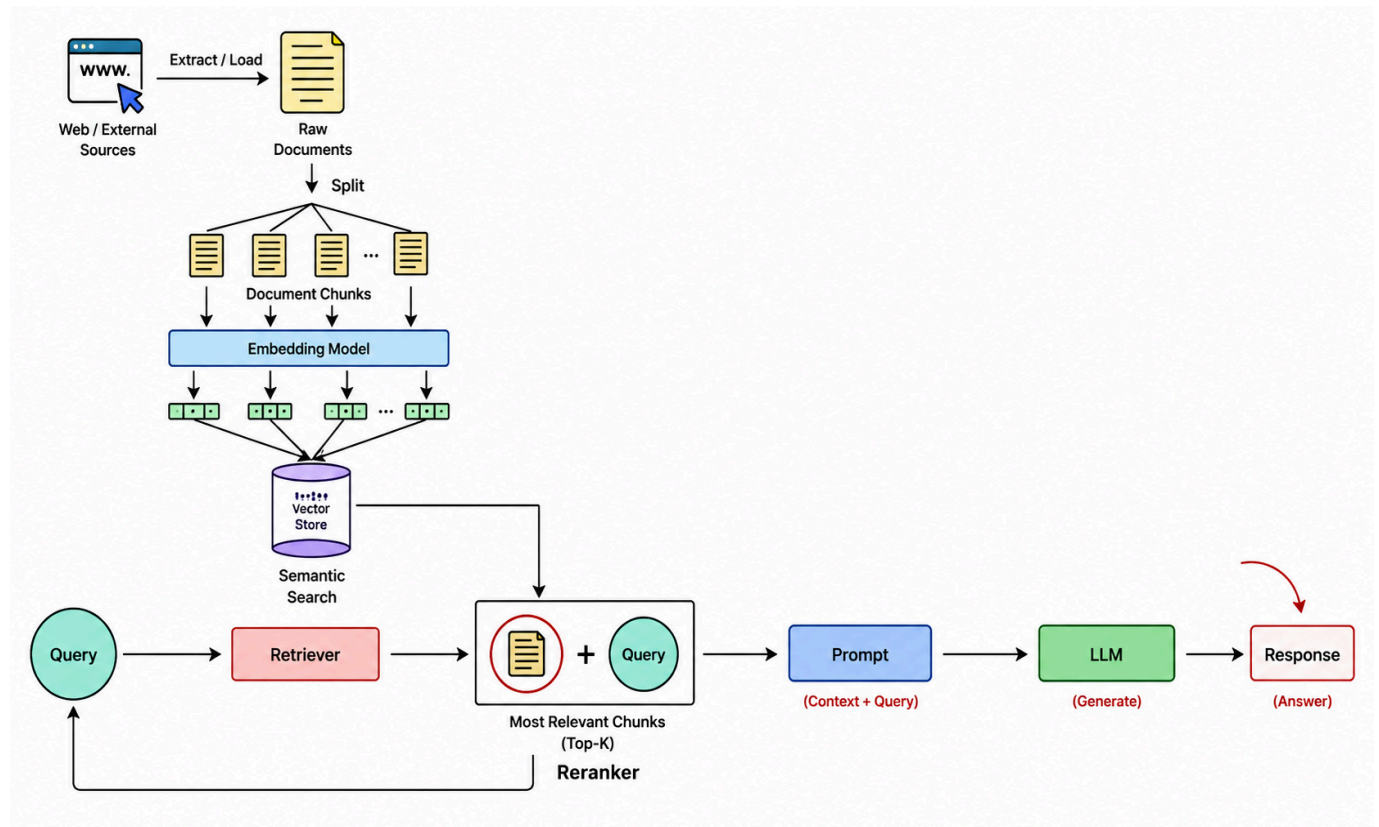
Problem	Description
Knowledge Cutoff	Models may not know facts or events that occurred after their training data.
Hallucination	Models can confidently generate incorrect or fabricated information.
No Private Data	Models don't automatically have access to your company's documents, databases, PDFs, or internal knowledge.
Context Window Limits	An entire large knowledge base cannot be included in a single prompt.
Costly Retraining	Updating a model's knowledge through fine-tuning can be expensive and time-consuming.

Why Do We Use RAG ?

RAG is useful because it allows an LLM to **retrieve relevant information from external sources before generating an answer.**

- **Up-to-date answers** - Retrieves the latest or most recent information.
- **Reduces hallucinations** - Grounds the model's response in retrieved facts and documents.
- **Private data access** - Lets the model use internal documents, databases, PDFs, and company knowledge.
- **Cheap to update** - Add or re-index new documents without retraining the LLM.
- **Traceable** - Responses can include citations showing which documents or sources were used.
- **Domain-specific** - Can be adapted to specialized fields such as **legal, medical, finance, and software development.**

How RAG Works



Before RAG what is used?

Fine-tuning: Fine-tuning trains the model on your specific dataset, making it useful for static and specialized tasks. However, it can be expensive and requires retraining when the knowledge changes.

Long-Context LLMs: Long-context models allow you to provide a large amount of information directly in the prompt. They are useful when the complete document or knowledge base can fit within the model's context window.

RAG Vs Fine Tuning

Feature	RAG	Fine-Tuning
Update Data?	Easy — just update the documents	Retraining needed
Real-time?	Yes	No

Private Docs?	Yes	Yes — knowledge is baked into model weights
Cost?	Low	High
Hallucination?	Reduced	Still possible
Transparency ?	Citable sources	Black box

Entra

What is Microsoft Entra?

Microsoft Entra is Microsoft’s family of identity and access management (IAM) products. The core service, Microsoft Entra ID, was formerly known as Azure Active Directory (Azure AD).

In simple terms, Entra ID is a cloud-based identity provider that verifies who you are and decides what you can access. It is used by millions of organizations to manage users, authenticate logins, and protect applications and data.

The broader Entra family includes additional products for identity governance, external identities, verifiable credentials, permissions management, and secure network access.

Concept	Meaning
Identity	A digital representation of a user, device, or service.
Authentication	Proving who you are (e.g., password, fingerprint, MFA).
Authorization	Deciding what you are allowed to do after authentication.
Tenant	A dedicated instance of Entra ID for an organization.
Directory	The database that stores users, groups, devices, and app registrations.
Single Sign-On (SSO)	Log in once and access many apps without re-entering credentials.
Multi-Factor Authentication (MFA)	Requiring two or more verification methods.
Conditional Access	Policies that enforce “if/then” rules for access (e.g., block login from risky locations).

The Microsoft Entra Family

Microsoft has expanded Entra beyond the core directory. The main products are:

- **Microsoft Entra ID** – Core identity and access management (formerly Azure AD).
- **Microsoft Entra ID Governance** – Automates user lifecycle, access reviews, and entitlement management.
- **Microsoft Entra External ID** – Customer identity and access management (CIAM) for external users, partners, and consumers.
- **Microsoft Entra Verified ID** – Decentralized identity and verifiable credentials based on open standards.
- **Microsoft Entra Permissions Management** – Cloud Infrastructure Entitlement Management (CIEM) to discover and manage permissions across AWS, Azure, and Google Cloud.
- **Microsoft Entra Internet Access** – Secure access to SaaS and internet resources (part of Microsoft's Security Service Edge).
- **Microsoft Entra Private Access** – Secure access to on-premises or private apps without a traditional VPN.
- **Microsoft Entra Suite** – A bundle that combines Entra ID, ID Governance, Verified ID, Internet Access, and Private Access.

Why Do We Use Microsoft Entra?

Organizations use Entra because modern IT is cloud-first, mobile, and hybrid. Traditional on-premises Active Directory is not designed for SaaS apps, cloud resources, and remote work. Entra ID fills that gap.

Key reasons:

1. Centralized identity management – One place to manage users, groups, devices, and app access.
2. Single Sign-On – Users access thousands of pre-integrated apps with one login.
3. Strong security – MFA, Conditional Access, and risk-based policies protect against identity attacks.
4. Zero Trust security model – Continuously verifies every access request.
5. Hybrid support – Works alongside on-premises Active Directory.
6. Collaboration – Securely invite external partners and customers.

7. Automation and governance – Automate user onboarding, offboarding, and access reviews.
8. Compliance – Detailed logs, reporting, and policy enforcement for regulatory requirements.

Identifying AI Agents (Agent Identities)

Microsoft Entra now includes a dedicated Agent identities feature for AI agents (e.g., Copilot Studio, Azure AI Foundry, Autonomous Agent). These are represented as service principals with an `agentIdentityBlueprint`.

A. Microsoft Entra admin center

1. Sign in to [Microsoft Entra admin center](#).
2. Go to Identity → Agents → Agent identities.
(Alternatively, search for “Agent identities” in the top bar.)
3. You can view:
 - Display name, Object ID
 - Agent type and blueprint (`agentIdentityBlueprint`)
 - Owners and sponsors
 - Permissions and Conditional Access policies

B. Microsoft Graph PowerShell

```
powershell
```

```
# Connect with required scope
```

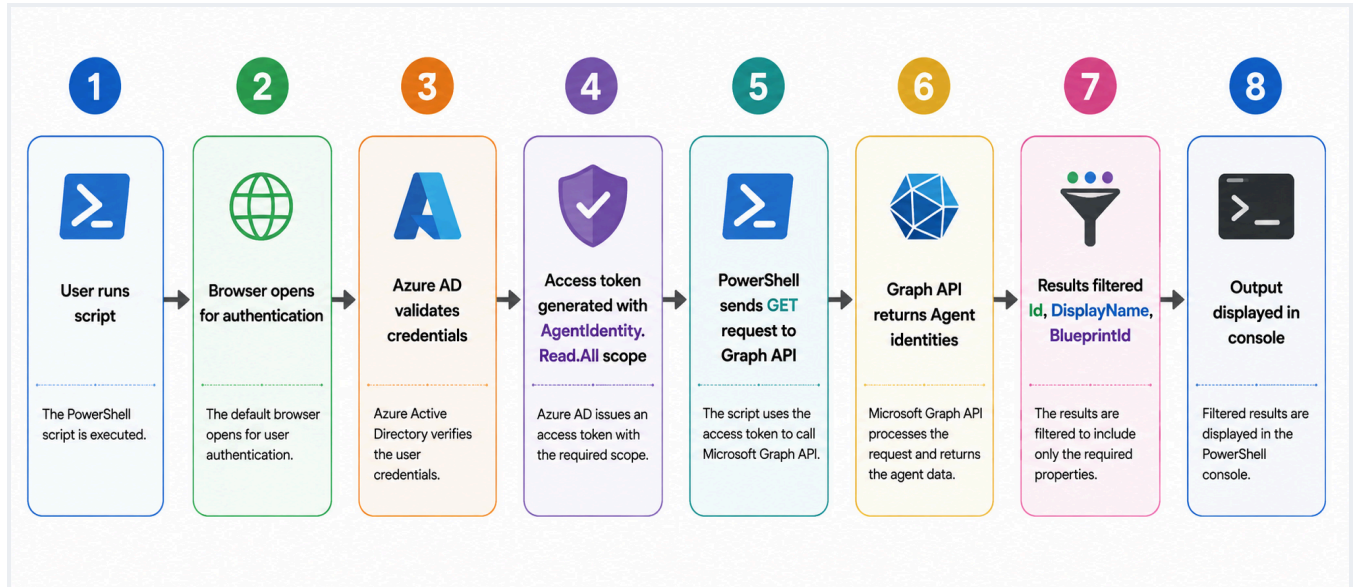
```
Connect-MgGraph -Scopes "AgentIdentity.Read.All"
```

```
# List all agent identities (beta cmdlet)
```

```
Get-MgBetaAgentIdentity | Select-Object Id, DisplayName, BlueprintId
```

Note: The cmdlet is in the beta module. Install it with:

```
Install-Module Microsoft.Graph.Beta -Scope CurrentUser
```



C. Microsoft Graph REST API (beta)

http

GET <https://graph.microsoft.com/beta/agentIdentities>

Authorization: Bearer {access-token}

Accept: application/json

How to get the access token

1. Register an app in Microsoft Entra ID.
2. Grant permission (API permission):
 - `AgentIdentity.Read.All` (application or delegated)
3. Acquire token using OAuth 2.0. For app-only (client credentials):

http

POST <https://login.microsoftonline.com/{tenant-id}/oauth2/v2.0/token>

Content-Type: application/x-www-form-urlencoded

```
client_id={client-id}

&client_secret={client-secret}

&scope=https://graph.microsoft.com/.default

&grant_type=client_credentials
```

- 4. Call the endpoint with the token in the **Authorization** header.

Important: This endpoint is in beta and may change. Ensure your tenant has the feature enabled (currently rolling out). Check the [Microsoft Graph documentation](#) for the latest permissions.

2. Identifying Other Types of Agents

If you’re looking for infrastructure or sync agents, use these locations:

Agent type	Where to find in Entra admin center
Provisioning / Cloud Sync agents	Hybrid management → Microsoft Entra Connect → Cloud sync → Agents
Application Proxy connectors	Global Secure Access → Connect → Connectors (or Enterprise applications → Application proxy → Connectors)

Connect Health agents (Sync, AD FS, AD DS)	Hybrid management → Microsoft Entra Connect → Connect Health
--	--

These agents are typically installed on-premises and are not the same as AI agent identities.

3. Identifying Client Applications / User-Agent Strings in Sign-in Logs

To see which client apps or devices are signing in (including raw User-Agent strings):

1. Go to Identity → Monitoring & health → Sign-in logs.
2. Select a sign-in record.
3. Under Basic info:
 - Client app – e.g., Browser, Mobile Apps, Legacy clients.
 - Resource / Application – target app.
4. Under Device info:
 - Browser, OS, and raw User Agent string.

This helps identify how an agent (if it's a client app) authenticates, but it is not the same as listing registered "agent identities".

Types of Agent

1. Copilot Studio Agents

What they are

Copilot Studio agents are conversational AI assistants built using Microsoft Copilot Studio, a low-code/no-code platform. They are designed for business users and makers who want to create chat-based agents without deep data science or AI engineering skills.

How they are built

- Created in a visual, web-based authoring canvas.
- Can use generative AI capabilities backed by Azure OpenAI.
- Connected to data sources such as SharePoint, Dataverse, websites, and external APIs.
- Published to channels like Microsoft Teams, websites, mobile apps, or internal portals.
- Often use Power Automate flows to perform actions.

Identity and governance characteristics

From a Microsoft Entra perspective, Copilot Studio agents are usually associated with:

- User identity when acting on behalf of a human during conversations.
- Workload identity when the agent runs unattended or accesses backend systems.
- Power Platform environment security and Data Loss Prevention policies.
- Microsoft Entra ID groups and roles to control who can create, edit, publish, and use the agent.
- Managed Environments for additional governance, such as sharing limits and usage analytics.
- Conditional Access policies can apply when the agent is accessed through Microsoft 365 services.

Example use cases

- HR assistant answering policy questions.
 - IT helpdesk agent resetting passwords or creating tickets.
 - Sales assistant looking up customer data from CRM.
 - Employee onboarding bot guiding new hires.
-

2. Azure AI Foundry Agents

What they are

Azure AI Foundry agents are custom, developer-built AI agents created using Azure AI Foundry (formerly Azure AI Studio). These agents are more technical and code-oriented than Copilot Studio agents. They provide fine-grained control over models, tools, orchestration, and deployment.

How they are built

- Developed using the Azure AI Foundry SDK or portal.
- Built on models from Azure OpenAI, such as GPT-4, or other open models.
- Use tools like:
 - Function calling
 - Code interpreter
 - Custom APIs
 - Vector databases and retrieval systems
- Can be integrated into applications via APIs or hosted as endpoints.
- Often involve Python, C#, or other programming languages.

Identity and governance characteristics

Azure AI Foundry agents typically rely on:

- Managed identities or service principals in Microsoft Entra ID.

- Azure RBAC roles to grant access to Azure resources like Azure OpenAI, Storage, Search, and Key Vault.
- Microsoft Entra Conditional Access for workload identities to control access to resources.
- Azure Policy for enforcing security and compliance guardrails.
- Network security features such as private endpoints and virtual networks.
- Diagnostic logging and monitoring through Azure Monitor and Microsoft Sentinel.
- Content safety filters and responsible AI tooling.

Example use cases

- Custom support agent integrated into a company's own application.
 - Document analysis agent that reads contracts and extracts key terms.
 - Code generation or review agent used by development teams.
 - Multi-tool research agent that searches internal knowledge bases and external APIs.
-

3. Autonomous Agents

What they are

Autonomous agents are AI systems designed to operate with minimal human intervention. They can plan, make decisions, call tools, and complete multi-step tasks on their own. These agents may be built using various Microsoft technologies, including Copilot Studio, Azure AI Foundry, Semantic Kernel, or AutoGen.

How they work

- Triggered by events, schedules, or user requests.
- Use planning and reasoning to break down goals into steps.
- Call APIs, databases, and other services to take actions.
- Can run long-running workflows and recover from errors.
- May include human-in-the-loop approval steps for sensitive actions.

- Often use memory and context to improve over time.

Identity and governance characteristics

Autonomous agents present more complex identity and governance challenges because they act independently. Microsoft Entra provides:

- Workload identities with least-privilege permissions.
- Lifecycle management for agent identities, including access reviews and expiration.
- Conditional Access policies for workload identities to limit where and how the agent can authenticate.
- Privileged Identity Management if the agent requires elevated permissions temporarily.
- Continuous Access Evaluation to revoke access in real time if risk changes.
- Audit logs and monitoring to track agent actions and detect anomalies.
- Federated identity credentials for agents running outside Azure, such as in GitHub Actions or other cloud platforms.

Example use cases

- Invoice processing agent that automatically reads emails, extracts invoice data, and updates financial systems.
- Supply chain agent that monitors inventory levels and places orders when stock is low.
- Security operations agent that investigates alerts and creates tickets.
- Sales outreach agent that drafts and sends follow-up emails based on CRM changes.

Summary Comparison

Feature	Copilot Studio Agents	Azure AI Foundry Agents	Autonomous Agents
---------	--------------------------	----------------------------	----------------------

Primary audience	Business users / makers	Developers / data scientists	Developers / architects
Development style	Low-code / no-code	Code-first	Code-first or low-code
Platform	Microsoft Copilot Studio	Azure AI Foundry	Multiple Microsoft tools
Orchestration	Built-in, conversation-focused	Custom, API/tool-based	Event-driven, goal-based
Identity type	User + workload identity	Managed identity / service principal	Workload identity
Governance focus	Power Platform policies	Azure RBAC / Policy	Lifecycle, least privilege, monitoring
Human involvement	Usually interactive	Can be interactive or API-driven	Minimal human intervention